Soal 1 — Konsep Procedural Programming

A. Pengertian Procedural Programming

Procedural programming itu gaya atau paradigma penulisan kode yang fokus sama urutan langkah-langkah/instruksi secara runtut dari atas ke bawah. Pada pendekatan ini, kita memecah program menjadi fungsi-fungsi atau prosedur kecil (seperti void atau function) untuk menyelesaikan tugas tertentu. Bahasa pemrograman yang sering pakai paradigma ini contohnya **C**, **Pascal**, dan **Fortran**.

B. Pengelolaan Data pada Pemrograman Prosedural

Dalam pendekatan prosedural, data dan fungsi dipisah secara tegas:

- Data bersifat pasif**:** Data disimpan di dalam variabel (lokal atau global) atau struktur data. Data ini nggak bisa 'mengolah dirinya sendiri', melainkan hanya nunggu diproses oleh fungsi.
- Fungsi bekerja aktif: Fungsi atau prosedur yang berisi instruksi untuk mengambil data, mengolahnya, lalu mengembalikan hasilnya.
- Alur Akses Data: Kalau fungsi butuh data, variabel dikirim sebagai argumen/parameter. Tapi kalau variabelnya dideklarasikan secara global, semua fungsi dalam program bisa mengakses dan mengubah isi data tersebut secara langsung.

Soal 2 — Perbedaan Procedural Programming dan OOP

Perbedaan utama antara *Procedural Programming* dan *Object-Oriented Programming* (OOP) dapat dilihat dari beberapa aspek berikut:

1. Penyimpanan Data

- Procedural Programming: Data dan fungsi dipisah. Data cuma disimpan di variabel pasif yang bisa diakses sama fungsi lain.
- OOP: Data dan fungsi dibungkus jadi satu di dalam Objek (enkapsulasi). Data jadi atribut milik objek tersebut.

2. Pengorganisasian Kode

- Procedural Programming**:** Kode dipecah-pecah jadi fungsi atau prosedur berdasarkan tugas tertentu (*task-oriented*).
- OOP: Kode diorganisasikan ke dalam Class dan Object yang merepresentasikan entitas dunia nyata (*entity-oriented*).

3. Fokus Utama

- Procedural Programming: Fokus ke urutan langkah atau alur logika (*how to do it*).
- OOP: Fokus ke objek dan cara objek-objek tersebut saling berinteraksi (*what is being manipulated*).

4. Keamanan Data

- Procedural Programming: Relatif kurang aman, karena variabel global bisa diubah dari fungsi mana saja secara tidak sengaja (*side effect*).

- OOP: Lebih aman karena ada enkapsulasi. Akses data bisa dibatasi (*private/protected*) dan cuma bisa diakses lewat *method* tertentu (*getter/setter*).

Soal 3

Kode itu cuma fungsi biasa yang mengubah data langsung. Tidak pakai class atau objek.

Bayangkan begini: buku itu seperti kartu catatan yang isinya judul dan status “tersedia: ya/tidak”. Fungsi pinjam_buku itu seperti petugas perpustakaan yang:

1. Melihat kartunya: “Bukunya masih ada nggak?”
2. Kalau ada, dia coret jadi “tidak tersedia”, lalu bilang “Berhasil dipinjam” (True).
3. Kalau tidak ada, dia bilang “Maaf, gagal” (False) dan kartunya tidak diubah.

Kenapa kartunya bisa berubah setelah fungsi selesai?
Karena fungsi tidak membuat salinan kartu. Dia menerima kartu yang *asli*, jadi coretannya langsung terlihat di luar fungsi. Dictionary di Python memang bisa diubah-ubah (*mutable*) dan dikirim ke fungsi sebagai rujukan, bukan salinan.

Jawaban singkat untuk soalnya:

- Pendekatan: prosedural (data dan fungsi dipisah), dengan data berupa dictionary yang diubah langsung.
- Alasan teknis: dictionary itu mutable dan dikirim sebagai referensi, jadi perubahan di dalam fungsi ikut mengubah data aslinya.

Kelemahannya: siapa saja bisa mengubah buku["tersedia"] langsung tanpa lewat fungsi, jadi rawan salah. Kalau programnya makin besar, biasanya dipindah ke class (OOP) supaya lebih rapi dan aman.

Soal 4 — Kapan Menggunakan OOP

Pengembangan sistem perpustakaan dengan penambahan jenis koleksi baru (misal: E-book, Jurnal, Majalah) dan jenis pengguna baru (misal: Dosen, Mahasiswa, Anggota Luar) sangat cocok menggunakan OOP karena alasan-alasan berikut:

- Kemudahan Pewarisan (*Inheritance*): Kita cukup membuat satu *Parent Class* utama (misalnya Koleksi atau Pengguna), lalu jenis baru tinggal mewarisi sifat dasar dari *class* tersebut. Contohnya, Ebook dan Jurnal tinggal menuruni sifat dari Koleksi tanpa perlu menulis ulang struktur dari awal.
- Prinsip *Polymorphism* (Beda Perilaku, Satu Perintah): Setiap jenis pengguna biasanya punya aturan pinjam yang berbeda (misal: Dosen boleh pinjam 30 hari, Mahasiswa cuma 7 hari). Dengan *polymorphism*, kita cukup memanggil satu fungsi seperti pinjam(), dan sistem otomatis menyesuaikan logikanya berdasarkan objek penggunanya.
- Skalabilitas & Kemudahan *Maintenance*: Saat nanti ada kebutuhan nambah koleksi atau jenis pengguna baru lagi, kita tinggal buat *Class* baru tanpa perlu mengacak-acak atau berisiko merusak logika program lama yang sudah berjalan.

- Enkapsulasi & Keamanan Data: Data transaksi perpustakaan dan informasi sensitif pengguna dibungkus rapi di dalam objek, sehingga tidak mudah diakses atau diubah secara tak sengaja oleh bagian program lain.

Soal 5 — Analisis Kebutuhan

Tiga pertanyaan analisis kebutuhan untuk menentukan apakah program sebaiknya memakai OOP atau *Procedural Programming*, beserta penjelasan jawaban yang mengarahkan ke OOP:

1. Seberapa kompleks entitas dan data yang ada di dalam sistem?

- Pertanyaan Detail: *"Apakah sistem memiliki banyak entitas dunia nyata yang saling berhubungan dan punya aturan bisnis masing-masing?"*
- Jawaban yang Mengarahkan ke OOP: Jika jawabannya "Ya, sistem punya banyak entitas kompleks (seperti Pengguna, Koleksi, Transaksi)", maka OOP adalah pilihan tepat. Entitas dunia nyata tersebut dapat direpresentasikan secara alami menjadi *Class* dan *Object* di dalam kode.

2. Seberapa besar kemungkinan aplikasi bakal dikembangkan atau ditambah fiturnya di masa depan?

- Pertanyaan Detail: *"Apakah aplikasi ini hanya program sekali pakai (skrip sederhana), atau aplikasi besar yang akan terus di-update dengan fitur/kategori baru?"*
- Jawaban yang Mengarahkan ke OOP: Jika jawabannya **"Aplikasi bakal terus berkembang dan ditambah fiturnya"**, maka wajib pakai OOP. Karena modularitas OOP tinggi, menambah fitur baru cukup dengan menambah *Class* baru tanpa perlu membongkar struktur kode lama.

3. Apakah ada kemiripan perilaku atau sifat antar entitas yang bisa dikelompokkan?

- Pertanyaan Detail: *"Apakah ada komponen-komponen sistem yang punya variabel/fungsi yang mirip, tapi punya variasi aturan kerja tersendiri?"*
- Jawaban yang Mengarahkan ke OOP: Jika jawabannya "Ya, banyak kemiripan sifat antar elemen", maka OOP sangat diuntungkan. Kita bisa memakai fitur *Inheritance* dan *Polymorphism* agar tidak perlu menulis kode berulang (*Don't Repeat Yourself / DRY*).